

O Problema dos k -Centros: Algoritmo Exato via Busca Binária com Branch-and-Bound versus Heurística de Aproximação de Gonzalez

João Comini Cesar de Andrade Rafael Rehfeld Martins de Oliveira

[Pontifícia Universidade Católica de Minas Gerais | jccandrade@sga.pucminas.br]

[Pontifícia Universidade Católica de Minas Gerais | rafael.oliveira.1508947@sga.pucminas.br]

Pontifícia Universidade Católica de Minas Gerais (PUC Minas), Brasil.

Resumo. Este trabalho aborda o problema dos k -centros, uma tarefa clássica de *clustering* e localização de facilidades que consiste em escolher k vértices de um grafo de modo a minimizar a maior distância entre qualquer vértice e seu centro mais próximo. São implementadas e comparadas duas abordagens: um método exato, baseado em busca binária sobre os valores de distância combinada com um teste de viabilidade resolvido por *branch-and-bound* sobre Cobertura de Conjuntos, e a heurística de Gonzalez, uma 2-aproximação gulosa de *farthest-point clustering*. Os métodos foram avaliados nas 40 instâncias de p -medianas da OR-Library (100 a 900 vértices). Os resultados mostram que o método exato só é tratável para instâncias pequenas e, sobretudo, para valores baixos de k , tornando-se inviável (>5 minutos) já a partir de instâncias intermediárias, enquanto a heurística de Gonzalez resolve todas as instâncias em milissegundos, com fator de aproximação empírico médio de 1,50 (ou seja, um erro médio de aproximadamente 50% acima do raio ótimo), sempre dentro do limite teórico de 2,0 (100% de erro).

Keywords: Problema dos k -Centros, Algoritmos de Aproximação, Heurística de Gonzalez, Branch-and-Bound, Cobertura de Conjuntos, OR-Library.

1 Introdução

O problema dos k -centros é uma tarefa clássica de análise de dados, intimamente relacionada a técnicas de *clustering*. Dado um conjunto de pontos com distâncias definidas entre cada par e um inteiro positivo k , o objetivo é selecionar k pontos (chamados *centros*) de forma a minimizar a maior distância entre qualquer ponto do conjunto e o centro mais próximo a ele. Essa distância máxima é denominada *raio* da solução.

Esse problema aparece naturalmente em cenários de localização de facilidades – por exemplo, decidir em quais de um conjunto de hospitais devem ser instalados centros de tratamento especializados, de modo que nenhum hospital fique excessivamente distante do centro mais próximo – e também em tarefas de categorização, como segmentar consumidores, alunos ou usuários de uma rede social em perfis homogêneos. É sabido que a resolução exata do problema é computacionalmente difícil em instâncias grandes (o problema de decisão associado é NP-completo, por redução a Cobertura de Conjuntos), o que motiva o uso de algoritmos aproximados.

Neste trabalho, implementam-se e comparam-se duas abordagens distintas para o problema dos k -centros:

1. **Método exato** (KCentrosExato): busca binária sobre os valores candidatos de raio, combinada com um teste de viabilidade *exato*, resolvido por enumeração exaustiva com poda (*branch-and-bound*) sobre o problema de Cobertura de Conjuntos;
2. **Heurística aproximada** (KCentrosGonzalez): o algoritmo guloso de Gonzalez (*farthest-point traversal*), que garante fator de aproximação 2 em relação ao raio ótimo.

As duas implementações são avaliadas sobre as 40 instâncias de p -medianas da OR-Library (`pmed1.txt` a `pmed40.txt`), com tamanhos de 100 a 900 vértices, comparando-se eficácia (qualidade da solução, medida pelo fator de aproximação em relação ao raio ótimo informado no enunciado) e eficiência (tempo de execução), à medida que o tamanho da instância e o valor de k aumentam.

2 Referencial Teórico

2.1 Definição Formal do Problema

Dada uma coleção de pontos V com distâncias definidas para cada par pela função $d : V \times V \mapsto \mathbb{R}^+$, o problema dos k -centros consiste em encontrar um conjunto $C \subseteq V$, com $|C| \leq k$, de forma que a maior distância de um vértice $v \in V$ ao centro mais próximo seja mínima, isto é, minimizar

$$r(C) = \max_{v \in V} \min_{c \in C} d(v, c).$$

Fixados os centros C , cada vértice é implicitamente associado ao centro mais próximo, definindo uma partição de V em até k *clusters*.

2.2 Equivalência com Cobertura de Conjuntos e Dificuldade

Para um raio candidato r fixo, perguntar se existe C , $|C| \leq k$, tal que todo vértice está a distância $\leq r$ de algum centro em C , é exatamente uma instância do problema de Cobertura de Conjuntos: cada vértice u define o subconjunto $S_u = \{v \in V \mid d(u, v) \leq r\}$ de vértices que ele consegue cobrir caso seja escolhido como centro, e deseja-se saber se é possível cobrir V com no máximo k desses subconjuntos [3]. Como Cobertura de Conjuntos é NP-completo, o problema de decisão dos k -centros também o é, e, consequentemente, a versão de otimização é NP-difícil.

2.3 Redução da Otimização a uma Sequência de Decisões

Hochbaum e Shmoys mostram que o raio ótimo r^* pertence necessariamente ao conjunto finito de distâncias par a par $\{d(u, v) \mid u, v \in V\}$ [5]. Isso permite reduzir o problema de otimização a uma busca sobre esse conjunto ordenado de valores: para cada raio candidato, resolve-se o problema de decisão correspondente (viável ou não), e busca-se o menor raio viável – naturalmente, por busca binária, já que a viabilidade é monótona em r (se um raio é viável, qualquer raio maior também é).

2.4 Não-Aproximabilidade Abaixo de Fator 2

Hochbaum e Shmoys também demonstram que, a menos que $P = NP$, não existe algoritmo de aproximação para o problema dos k -centros com fator menor que 2 [5]. Esse resultado situa a heurística de Gonzalez, descrita a seguir, como assintoticamente ótima entre os algoritmos polinomiais conhecidos para o problema.

2.5 Heurística de Gonzalez (Farthest-Point Clustering)

Gonzalez propõe um algoritmo guloso que constrói os k centros iterativamente: o primeiro centro é escolhido arbitrariamente e, a cada passo seguinte, escolhe-se como novo centro o vértice com a maior distância mínima ao conjunto de centros já selecionado [4]. Esse procedimento garante que o raio final obtido é, no máximo, o dobro do raio ótimo r^* – a prova segue de um argumento de pombos: se houvesse $k + 1$ pontos mutuamente a distância $> 2r^*$, pela desigualdade triangular nenhum centro ótimo poderia cobrir dois deles simultaneamente com raio r^* , contradizendo a existência de uma solução ótima com apenas k centros.

2.6 Floyd-Warshall

Como as instâncias da OR-Library fornecem um grafo esparsos com pesos nas arestas (e não diretamente a matriz de distâncias completa exigida pela formulação do problema), é necessário pré-processar as distâncias de menor custo entre todos os pares de vértices. Isso é feito com o algoritmo de Floyd-Warshall, de complexidade $O(n^3)$ [2].

3 Metodologia

3.1 Instâncias de Teste

Conforme exigido pelo enunciado do trabalho, as implementações foram testadas com as 40 instâncias do problema (não capacitado) das p -medianas da OR-Library (`pmed1.txt` a `pmed40.txt`) [1]. Embora o problema das p -medianas use uma função objetivo diferente (soma das distâncias, em vez do máximo), a mesma estrutura de grafo é reaproveitada como entrada para o problema dos k -centros, conforme indicado no enunciado. Cada arquivo contém, na primeira linha, o número de vértices n , o número de arestas m e o número de centros k , seguidos por m linhas no formato (u, v, peso) . A Tabela 1 resume as características das instâncias e os valores de raio ótimo fornecidos no enunciado, usados como referência para o cálculo do fator de aproximação.

3.2 Algoritmo Exato

O método exato realiza uma busca binária sobre os valores distintos de distância entre pares de vértices (obtidos

Table 1: Faixas de tamanho das 40 instâncias utilizadas.

$ V $	Instâncias	k (faixa)	Raio ótimo (faixa)
100–900	pmed1–pmed40	5–200	9–127

após Floyd-Warshall). Para cada raio candidato r , testa-se a viabilidade resolvendo, de forma exaustiva, o problema de Cobertura de Conjuntos correspondente: a cada passo, escolhe-se o vértice ainda descoberto com **menor número de candidatos capazes de cobri-lo** (heurística *most-constrained-first*, usada apenas para reduzir o fator de ramificação médio – sem comprometer a correção do resultado) e ramifica-se sobre cada um desses candidatos, com poda quando o número de centros já utilizados atinge k sem cobrir todos os vértices. Por não empregar nenhuma heurística na decisão de viabilidade em si, o método é garantidamente exato, ao custo de complexidade exponencial no pior caso.

3.3 Heurística de Gonzalez

A heurística mantém, para cada vértice v , a distância mínima $\text{distMin}[v]$ até o conjunto de centros escolhido até o momento. O primeiro centro é o vértice de índice 0; a cada iteração subsequente, escolhe-se o vértice que maximiza $\text{distMin}[v]$ como novo centro, e o vetor é atualizado em $O(n)$. Após a escolha dos k centros, o raio reportado é $\max_v \text{distMin}[v]$.

3.4 Métricas e Protocolo Experimental

Para cada instância e método, foram registrados o **raio obtido**, o **fator de aproximação** ($FA = \text{raio obtido} / \text{raio ótimo}$) e o **tempo de execução** (em segundos). Como o método exato sempre encontra o raio ótimo quando conclui, o FA só é informativo para a heurística de Gonzalez; nesse caso, ele também é expresso como um **erro percentual** $\text{Erro}(\%) = (FA - 1) \times 100\%$, isto é, o quanto o raio aproximado excede o raio ótimo (limite teórico de Gonzalez: $FA \leq 2 \Leftrightarrow \text{Erro} \leq 100\%$).

A heurística de Gonzalez é sempre executada, em todas as 40 instâncias. Já o método exato é executado em uma *thread* separada com um limite de tempo de 300 segundos (5 minutos); caso o limite seja excedido, a execução é interrompida e o resultado é descartado – no `resultados.csv`, essas instâncias aparecem marcadas com `-1` nas colunas `exato_raio`, `exato_fa` e `exato_tempo_s`, indicando que o método exato **não conseguiu terminar dentro do limite de tempo** para aquela instância, e que, portanto, somente a heurística de Gonzalez fornece uma solução (aproximada) nesses casos.

4 Implementação

A implementação foi realizada em Java, organizada em três classes principais: `KCentrosExato.java`, `KCentrosGonzalez.java` e `Experimentos.java`, além das 40 instâncias da OR-Library na pasta `instancias/`.

4.1 Leitura de Instâncias e Pré-processamento

O método `lerInstancia` (idêntico nas duas classes de solução) lê o cabeçalho (n, m, k) e as m arestas no

formato da OR-Library, montando uma matriz de adjacência inicializada com ∞ (e 0 na diagonal). Em seguida, `floydWarshall` calcula a matriz de distâncias de menor custo entre todos os pares, em $O(n^3)$, necessária pois a formulação do problema dos k -centros pressupõe um grafo completo de distâncias.

4.2 KCentrosExato: Busca Binária + Branch-and-Bound

O método `resolve()` coleta os valores distintos de distância, ordena-os e realiza busca binária, chamando `verificaViabilidadeExata(r)` para cada raio candidato. Essa função constrói, para cada vértice v , a lista de candidatos que o cobrem com raio r , e delega a busca para `buscaExaustiva`, que implementa o *branch-and-bound* sobre Cobertura de Conjuntos descrito a seguir.

Listing 1: Esqueleto do branch-and-bound (most-constrained-first).

```
1 buscaExaustiva(coberto, centros, total):
2   se total == n: retorna centros (solucao)
3   se |centros| == k: retorna null (poda)
4   v := vertice descoberto com menos
5       candidatos que o cobrem
6   para cada candidato de cobrePor[v]:
7     aplica candidato (marca cobertos)
8     resultado := buscaExaustiva(...)
9     desfaz marcacao (backtracking)
10  se resultado != null: retorna resultado
11  retorna null
```

A escolha do vértice mais restrito antes de ramificar (linhas 4–5) não altera a correção – esse vértice precisa ser coberto de qualquer forma – mas reduz substancialmente o fator de ramificação médio na prática, por podar cedo os ramos inviáveis.

4.3 KCentrosGonzalez: Farthest-Point Greedy

A implementação segue diretamente o algoritmo de Gonzalez: um laço de $k - 1$ iterações que, a cada passo, varre o vetor `distMin` em $O(n)$ para encontrar o próximo centro e atualizá-lo, resultando em complexidade total $O(k \cdot n)$ após o pré-processamento de Floyd-Warshall.

4.4 Experimentos.java: Orquestração

A classe `Experimentos` percorre as 40 instâncias, executa sempre a heurística de Gonzalez e, para o método exato, inicia uma `Thread` dedicada e aguarda sua conclusão por até `LIMITE_MS = 300_000` milissegundos (`worker.join(LIMITE_MS)`); se a `thread` ainda estiver viva após esse prazo, ela é interrompida e o resultado é descartado. Os raios ótimos de referência (Tabela 1 do enunciado) estão embutidos no vetor `RAIOS_OTIMOS` e são usados apenas para calcular o fator de aproximação de cada método. Todos os resultados são impressos em formato tabular e gravados em `resultados.csv`.

5 Experimentos e Resultados

Os experimentos foram executados sobre as 40 instâncias `pmed1–pmed40` (n entre 100 e 900, k entre 5 e 200). A Tabela 2 apresenta os resultados completos: raio ótimo de referência, raio e tempo obtidos pelo método exato (quando concluído dentro do limite de 300s) e pela heurística de Gonzalez.

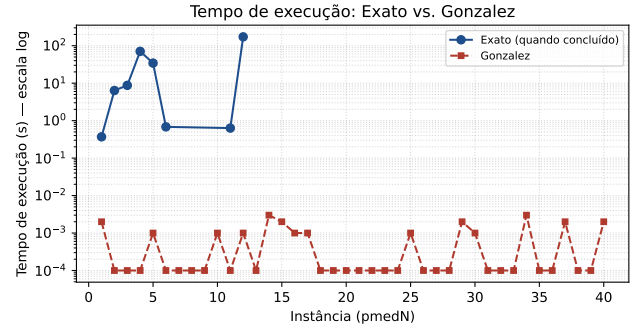


Figure 1: Tempo de execução (escala log) do método exato (apenas instâncias concluídas) e da heurística de Gonzalez, por instância.

5.1 Desempenho do Método Exato

O método exato concluiu dentro do limite de 300s em apenas 8 das 40 instâncias: `pmed1–5` ($n = 100$), `pmed6` e `pmed11` ($n = 200$ e $n = 300$, ambas com $k = 5$) e `pmed12` ($n = 300$, $k = 10$). Em todos os casos concluídos, o raio obtido coincide exatamente com o raio ótimo de referência ($FA = 1,00$), confirmando a correção da implementação. A Figura 1 ilustra, em escala logarítmica, o crescimento abrupto do tempo do método exato.

É notável que o tempo de execução do método exato não cresce de forma monotônica apenas com n : a instância `pmed4` ($n = 100$, $k = 20$) levou 70,7s, mais do que `pmed6` ($n = 200$, $k = 5$, apenas 0,68s). O fator dominante é o valor de k – quanto maior o número de centros a escolher, maior a profundidade da árvore de busca do *branch-and-bound* e maior o número de combinações possíveis de centros a explorar em cada teste de viabilidade. Esse efeito é ainda mais evidente ao comparar `pmed11` ($n = 300$, $k = 5$, 0,63s) com `pmed12` ($n = 300$, $k = 10$, 172,4s): dobrar k de 5 para 10, mantendo n fixo, multiplicou o tempo de execução por mais de 270 vezes. Já para $k \geq 30$ (`pmed13` em diante) ou para $k = 10$ com $n \geq 400$ (`pmed17`), o método exato não concluiu dentro do limite de 5 minutos em nenhum dos testes restantes, evidenciando a complexidade exponencial no pior caso do teste de viabilidade.

5.2 Desempenho da Heurística de Gonzalez

Em contraste direto, a heurística de Gonzalez resolveu todas as 40 instâncias – incluindo a maior, com 900 vértices – em no máximo 3 milissegundos, refletindo sua complexidade polinomial $O(k \cdot n)$ após o pré-processamento. O fator de aproximação variou entre 1,24 (`pmed11`) e 1,79 (`pmed16`), com média empírica de 1,50 e desvio padrão de 0,12 – portanto, sempre confortavelmente abaixo do limite teórico de 2,0 garantido por Gonzalez [4], como mostra a Figura 2. Em termos de erro percentual em relação ao raio ótimo (coluna $Erro_G$ da Tabela 2), isso significa que a heurística errou, em média, 49,6% acima do ótimo, com mínimo de 23,7% (`pmed11`) e máximo de 78,7% (`pmed16`) – sempre dentro do limite teórico de 100% de erro ($FA = 2$). Não há tendência clara de piora do fator de aproximação (ou do erro percentual) com o crescimento de n ou k nas instâncias testadas, sugerindo que o comportamento médio da heurística é consideravelmente melhor do que sua garantia de pior caso para estas instâncias da OR-Library.

Table 2: Resultados completos para as 40 instâncias (raio ótimo conforme enunciado; “–” indica que o método exato **não concluiu em 300s** – nesses casos, o `resultados.csv` traz `-1`, e apenas a heurística de Gonzalez resolve a instância. $\text{Erro}_G = (\text{FA}_G - 1) \times 100\%$ é o quanto o raio de Gonzalez excede o raio ótimo.).

Inst.	n	k	Raio ót.	Exato	FA_E	$T_E(\text{s})$	Gonzalez	FA_G	Erro_G	$T_G(\text{s})$
1	100	5	127	127	1,00	0,370	186	1,46	46,5%	0,002
2	100	10	98	98	1,00	6,371	131	1,34	33,7%	0,000
3	100	10	93	93	1,00	8,751	154	1,66	65,6%	0,000
4	100	20	74	74	1,00	70,688	114	1,54	54,0%	0,000
5	100	33	48	48	1,00	34,352	71	1,48	47,9%	0,001
6	200	5	84	84	1,00	0,683	138	1,64	64,3%	0,000
7	200	10	64	–	–	–	96	1,50	50,0%	0,000
8	200	20	55	–	–	–	82	1,49	49,1%	0,000
9	200	40	37	–	–	–	57	1,54	54,0%	0,000
10	200	67	20	–	–	–	31	1,55	55,0%	0,001
11	300	5	59	59	1,00	0,633	73	1,24	23,7%	0,000
12	300	10	51	51	1,00	172,426	71	1,39	39,2%	0,001
13	300	30	35	–	–	–	59	1,69	68,6%	0,000
14	300	60	26	–	–	–	40	1,54	53,8%	0,003
15	300	100	18	–	–	–	25	1,39	38,9%	0,002
16	400	5	47	–	–	–	84	1,79	78,7%	0,001
17	400	10	39	–	–	–	56	1,44	43,6%	0,001
18	400	40	28	–	–	–	44	1,57	57,1%	0,000
19	400	80	18	–	–	–	28	1,56	55,6%	0,000
20	400	133	13	–	–	–	19	1,46	46,2%	0,000
21	500	5	40	–	–	–	53	1,32	32,5%	0,000
22	500	10	38	–	–	–	56	1,47	47,4%	0,000
23	500	50	22	–	–	–	34	1,55	54,6%	0,000
24	500	100	15	–	–	–	23	1,53	53,3%	0,000
25	500	167	11	–	–	–	15	1,36	36,4%	0,001
26	600	5	38	–	–	–	50	1,32	31,6%	0,000
27	600	10	32	–	–	–	43	1,34	34,4%	0,000
28	600	60	18	–	–	–	28	1,56	55,6%	0,000
29	600	120	13	–	–	–	19	1,46	46,2%	0,002
30	600	200	9	–	–	–	14	1,56	55,6%	0,001
31	700	5	30	–	–	–	42	1,40	40,0%	0,000
32	700	10	29	–	–	–	45	1,55	55,2%	0,000
33	700	70	15	–	–	–	25	1,67	66,7%	0,000
34	700	140	11	–	–	–	17	1,55	54,6%	0,003
35	800	5	30	–	–	–	38	1,27	26,7%	0,000
36	800	10	27	–	–	–	41	1,52	51,8%	0,000
37	800	80	15	–	–	–	25	1,67	66,7%	0,002
38	900	5	29	–	–	–	39	1,34	34,5%	0,000
39	900	10	23	–	–	–	35	1,52	52,2%	0,000
40	900	90	13	–	–	–	21	1,62	61,5%	0,002

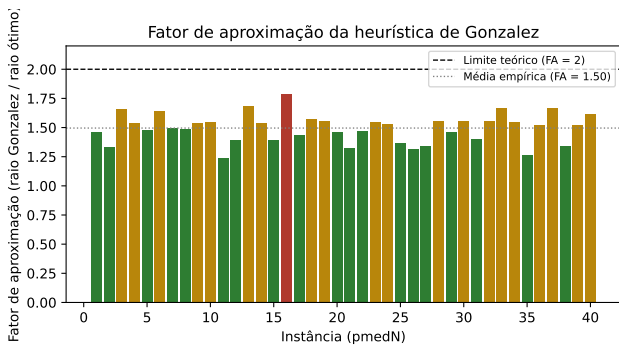


Figure 2: Fator de aproximação da heurística de Gonzalez em cada uma das 40 instâncias, com a média empírica e o limite teórico de 2,0.

5.3 Análise Comparativa

Os resultados evidenciam o compromisso clássico entre exatidão e escalabilidade. O método exato garante a solução ótima ($\text{FA} = 1$), mas seu custo exponencial restringe sua aplicabilidade prática a instâncias pequenas e, principal-

mente, a valores baixos de k – mesmo instâncias com n moderado (300 vértices) tornam-se intratáveis quando k cresce. Esse comportamento é consistente com a natureza do teste de viabilidade: o número de subconjuntos de centros de tamanho k a considerar cresce combinatorialmente com k , e a busca binária sobre o raio multiplica esse custo por $O(\log n^2)$ testes de viabilidade.

Por outro lado, a heurística de Gonzalez escala perfeitamente para todas as instâncias testadas, sacrificando otimalidade por uma garantia de qualidade constante (fator 2) que, na prática, se mostrou bem mais apertada ($\approx 1,5$ em média). Para aplicações reais envolvendo grafos de médio a grande porte ou valores de k moderados a altos, a heurística de Gonzalez é claramente a escolha mais viável; o método exato permanece útil como referência de qualidade (*ground truth*) para validar heurísticas em instâncias pequenas.

6 Conclusão

Este trabalho implementou e comparou duas abordagens para o problema dos k -centros: um método exato baseado em busca binária com teste de viabilidade por *branch-and-bound*

sobre Cobertura de Conjuntos, e a heurística gulosa de Gonzalez, com garantia teórica de 2-aproximação. Os experimentos sobre as 40 instâncias de p -medianas da OR-Library confirmam que o método exato, apesar de garantir a solução ótima, é inviável na prática assim que k ultrapassa valores pequenos (em torno de 10), independentemente do tamanho n da instância, em razão da complexidade exponencial do teste de viabilidade subjacente. Já a heurística de Gonzalez resolveu todas as instâncias – inclusive a de 900 vértices – em tempo desprezível, com fator de aproximação empírico médio de 1,50 (erro médio de 49,6% acima do raio ótimo), sempre respeitando o limite teórico de 2,0 (100% de erro). Conclui-se que, para aplicações práticas em larga escala, a heurística de Gonzalez é a opção recomendada, reservando-se o método exato para validação em instâncias pequenas ou com poucos centros.

Disponibilidade de dados e materiais

O código-fonte com todas as implementações (KCentrosExato.java, KCentrosGonzalez.java, Experimentos.java), as instâncias utilizadas e os resultados brutos (resultados.csv) acompanham a entrega deste trabalho.

References

- [1] Beasley, J. E. (1990). OR-Library: Distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11), 1069–1072. DOI: <https://doi.org/10.1057/jors.1990.166>
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. The MIT Press, third edition.
- [3] Garey, M. R., and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.
- [4] Gonzalez, T. F. (1985). Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38, 293–306. DOI: [https://doi.org/10.1016/0304-3975\(85\)90224-5](https://doi.org/10.1016/0304-3975(85)90224-5)
- [5] Hochbaum, D. S., and Shmoys, D. B. (1985). A best possible heuristic for the k -center problem. *Mathematics of Operations Research*, 10(2), 180–184. DOI: <https://doi.org/10.1287/moor.10.2.180>